

Student Guide

Chess Agent Competition

You will write one Python program that plays chess under a fixed resource budget. Course staff will evaluate valid agents against hidden benchmarks and against the other valid student agents.

Read the Project Information Sheet first for the learning objectives, deliverables, grading framework, and timeline. This guide explains how to install the starter package, implement the public API, test an agent locally, and submit the required file. The [Public Competition Rules](#) are the authoritative source for official clocks, fault handling, scoring, tiebreaks, and hidden or public information.

1. The idea

Your agent is a resource-bounded strategy:

```
observation -> decision rule -> legal UCI move
```

A strong submission should return legal moves reliably, use its limited clock effectively, and implement a method that you can explain and evaluate in your final report. Tournament rank is one part of the project. Methodology, experiments, game-theoretic analysis, and written communication are also graded.

2. Setup and first run

Install `uv` from the official Astral documentation, then extract the starter ZIP to a normal writable folder. Do not run the package from inside the ZIP. On Windows, prefer a short path such as `C:\ChessTournament` because legacy package-build tools may fail under unusually deep directory paths.

Key files in the extracted package are:

```
Chess_Agent_Tournament_Student_v1.1/
├── START_HERE_WINDOWS.cmd
├── START_HERE_MACOS_LINUX.sh
├── README_FIRST.md
├── .python-version
├── pyproject.toml
├── uv.lock
├── chess_tournament_api/      public API; do not modify
├── chess_student_app/        local validator and match application
├── public_agents/            Random and Greedy practice opponents
├── workspace/
│   ├── TEMPLATE_agent.py
│   └── example_minimax.py
├── docs/
│   ├── PROJECT_DESCRIPTION.pdf
│   ├── STUDENT_GUIDE.pdf
│   └── PUBLIC_COMPETITION_RULES.md
```

On Windows, double-click `START_HERE_WINDOWS.cmd`.

On macOS or Linux, open a terminal in the extracted folder and run:

```
chmod +x START_HERE_MACOS_LINUX.sh
./START_HERE_MACOS_LINUX.sh
```

The launchers run the equivalent of:

```
uv sync --frozen
uv run python -m chess_student_app
```

The first launch installs the pinned Python runtime, creates `.venv` inside the extracted folder, and installs the exact packages in `uv.lock`. Later launches reuse that environment. Do not copy a `.venv` from another computer.

Chess Agent Test Space opens in your browser and runs on your computer at `127.0.0.1`. Use it to check your file and play practice games. Submit your final file separately through Gradescope. Close the terminal window or press `Ctrl+C` to stop the app.

Copy `workspace/TEMPLATE_agent.py` to `workspace/YOUR_ACTUAL_NETID.py`. Keep the untouched template so that you can compare your file with a minimal working implementation.

3. The agent interface

Your submission must define exactly one class derived from `ChessAgent`.

```
import chess

from chess_tournament_api import ChessAgent, GameResult, Observation

class MyAgent(ChessAgent):
    name = "My Strategic Agent"

    def initialize(
        self,
        *,
        color: str,
        opponent_id: str,
        time_control: tuple[float, float],
        seed: int,
    ) -> None:
        """Optional. Called at the beginning of each game."""
        pass

    def choose_move(self, observation: Observation) -> str:
        """Required. Return one legal move in UCI notation."""
        board = chess.Board(observation.fen)
        return next(iter(board.legal_moves)).uci()

    def on_game_end(self, result: GameResult) -> None:
        """Optional. Called after a completed game."""
        pass
```

Only `choose_move` is required. It must return a built-in Python `str` containing one legal UCI move. A normal response uses four or five ASCII characters and may not exceed the published response limit.

`initialize` and `on_game_end` are optional, time-limited callbacks. They can initialize seeded randomness, reset per-game state, or update bounded information about the same anonymous opponent. Your class must also be constructible without arguments:

```
agent = MyAgent()
```

The class-level `name` is your public tournament alias. Your NetID filename remains the authoritative private submission identity.

4. Observations, results, and matchup memory

`choose_move` receives an immutable `Observation`.

Field	Type	Meaning
<code>fen</code>	<code>str</code>	Current position in FEN notation
<code>move_history</code>	<code>tuple[str, ...]</code>	Moves already played in the current game, in UCI
<code>my_remaining_time</code>	<code>float</code>	Your remaining chess clock in seconds
<code>opponent_remaining_time</code>	<code>float</code>	The opponent's remaining clock in seconds
<code>increment</code>	<code>float</code>	Time added after each accepted legal move
<code>color</code>	<code>str</code>	"white" or "black"
<code>opponent_id</code>	<code>str</code>	Anonymous identifier stable only within this matchup
<code>match_game_index</code>	<code>int</code>	One-based game number within this matchup or series
<code>seed</code>	<code>int</code>	Reproducible per-game seed

Build the board from the FEN. The board already knows whose turn it is:

```
board = chess.Board(observation.fen)
```

`on_game_end` receives an immutable `GameResult` written from your agent's perspective.

Field	Type	Meaning
<code>game_id</code>	<code>str</code>	Opaque identifier for the completed game
<code>opponent_id</code>	<code>str</code>	Matchup-local anonymous opponent identifier
<code>my_color</code>	<code>str</code>	Your color in the game
<code>outcome</code>	<code>str</code>	"win", "draw", or "loss"
<code>my_score</code>	<code>float</code>	Your chess points from the game
<code>opponent_score</code>	<code>float</code>	The opponent's chess points from the game
<code>termination_code</code>	<code>str</code>	Normalized reason the game ended
<code>fault_side</code>	<code>str</code> or <code>None</code>	"self", "opponent", or <code>None</code>
<code>moves</code>	<code>tuple[str, ...]</code>	Complete UCI move list
<code>my_remaining_time</code>	<code>float</code>	Your remaining clock at game end
<code>opponent_remaining_time</code>	<code>float</code>	Opponent's remaining clock at game end
<code>starting_fen</code>	<code>str</code>	Starting position used for the game
<code>seed</code>	<code>int</code>	Reproducible per-game seed
<code>match_game_index</code>	<code>int</code>	One-based game number within the matchup

These objects never disclose a real name, NetID, internal participant ID, opponent alias, or future schedule.

A fresh agent instance is created for each benchmark matchup, student-league matchup, or playoff series. The same instance may persist across the games of that matchup, and `opponent_id` remains stable within it. The process and all state are destroyed after the matchup. An agent may therefore adapt to the same anonymous opponent across a series, but may not carry tournament-wide memory to a later opponent.

Do not start a background thread or process to think during the opponent's turn. The official system may suspend your worker whenever a callback is not active.

5. Move format

Return moves in UCI notation.

Move	UCI string
Pawn e2 to e4	e2e4
Knight g1 to f3	g1f3
White castles kingside	e1g1
En passant e5 to d6	e5d6
Pawn promotes to queen	e7e8q

The safest method is:

```
return move.uci()
```

Do not return a `chess.Move` object, SAN, or descriptive text.

6. Submission format

Upload exactly one Python file to the designated Gradescope assignment:

```
YOUR_ACTUAL_NETID.py
```

For example, a student with NetID `abc123` submits `abc123.py`. Do not submit the starter ZIP, `.venv`, a folder, or a source archive.

Your file must:

- be no larger than 1 MiB;
- define exactly one eligible `ChessAgent` subclass;
- construct with no arguments;
- provide a valid public alias and implement `choose_move`;
- return a legal UCI string for every tested position;
- use only the Python standard library and the supplied `chess==1.11.2` dependency; and
- contain all authorized logic needed by the agent.

Do not use an external engine, network service, subprocess, compiled binary, additional model or data file, or another student's code. Imports associated with networking, FFI, and child processes are rejected by the local dependency check and prohibited by the official rules.

The public alias must contain 1-40 ASCII letters, digits, spaces, periods, underscores, or hyphens, and its first character must be a letter or digit. Duplicate valid aliases may receive an instructor-added display suffix.

7. Local validation and matches

In Chess Agent Test Space, follow the four steps:

1. **Add Your Agent:** choose your Python file and select **Add File**.
2. **Choose the Agent to Test:** select the file you want to run.
3. **Validate Your Agent:** select **Run Validation**. Each check shows PASS or FAIL; a failed check explains what to fix. Your final file must show **READY TO SUBMIT**.
4. **Run Test Games:** choose Random or Greedy and the number of games, then select **Run Games**. When finished, select **View Game Report**.

The web page shows short check results. You can expand **Technical Details** on a failed check or use the CLI for more diagnostic information:

```
uv run python -m chess_student_app.cli validate workspace/YOUR_ACTUAL_NETID.py
```

The validator checks the filename, file size, syntax, dependencies, subclass count, no-argument construction, alias, legal string responses in both colors against Random and Greedy, callback faults, and seeded replay behavior.

The untouched `TEMPLATE_agent.py` is an installation check and prints:

```
DEVELOPMENT CHECK PASSED — NOT SUBMISSION READY
```

A correctly named submission that passes the tested contract prints:

```
PASSED — SUBMISSION READY
```

This result means that the file passed the submission checks in the tested cases. It does not guarantee strategic strength, playoff qualification, or correctness on every possible chess position. The web page reports this result as **READY TO SUBMIT**.

Use the local app or CLI to compare versions:

```
uv run python -m chess_student_app.cli match --agent workspace/YOUR_ACTUAL_NETID.py --
opponent random --games 20
uv run python -m chess_student_app.cli match --agent workspace/YOUR_ACTUAL_NETID.py --
opponent greedy --games 20
```

You may also compare two of your own files:

```
uv run python -m chess_student_app.cli match --agent workspace/YOUR_ACTUAL_NETID.py --
opponent-file workspace/my_old_agent.py --games 20
```

Game reports identify each side by its file name and color, show your wins, draws, losses, errors, and time per move, and offer PGN downloads. R1, R2, and later labels group each two-game color swap; an odd game count leaves the last group with one game. In Replay, drag the horizontal slider or use Previous, Next, or the left and right arrow keys to move through the game. Each position shows the color, standard chess notation, and a short move description. **Final time** is the clock remaining at the end of the game. Practice results are shown here; official results are calculated after submission.

8. Official competition summary

Under Public Rules version 1.2, each valid agent receives:

$$\begin{aligned} B &= 100 * \text{benchmark points} / 32 \\ R &= 100 * \text{league points} / [4(N - 1)] \\ Q &= 0.5B + 0.5R \end{aligned}$$

B is based on 32 games against four hidden benchmark variants. R is based on four games against every other valid student agent. Each student pair uses two frozen legal starting positions, with one game in each color from each position. For one valid submission, $Q = B$ and no championship bracket is held.

The highest Q scores advance to a seeded playoff bracket:

Valid agents N	Playoff field K
$N \geq 32$	16
$16 \leq N < 32$	8
$8 \leq N < 16$	4
$2 \leq N < 8$	2
$N < 2$	No championship bracket

The playoffs determine the class Champion. Exact game counts, qualification and playoff tiebreaks, clocks, fault handling, final placement, and Tournament Performance formulas are defined in `PUBLIC_COMPETITION_RULES.md`. If this summary and the Public Competition Rules ever differ, the Public Competition Rules control.

9. Strategy and experiment ideas

Start with reliability. Keep a guaranteed legal fallback move even when a more ambitious search runs out of time.

- **Evaluation:** compare material-only evaluation with mobility, king safety, pawn structure, center control, or piece-square terms.
- **Search:** test minimax or negamax with alpha-beta pruning, move ordering, quiescence search, or a transposition table that fits the resource rules.
- **Time management:** iterative deepening can preserve the best completed result when the clock becomes tight. Allocate less time to forced moves and more to tactically sharp positions.
- **Seeded randomization:** break equal-valued choices using only the supplied seed. This can reduce predictability while keeping games reproducible.
- **Matchup-local adaptation:** record bounded features from earlier games against the same anonymous opponent and test whether adaptation improves a series.

Useful controlled comparisons include move ordering on versus off, fixed depth versus iterative deepening, material-only versus positional evaluation, quiescence on versus off, deterministic versus seeded tie-breaking, and adaptation on versus off. Run enough repeated games to distinguish a stable effect from one lucky result, and report both strength and reliability.

10. Common mistakes

- Returning `chess.Move` instead of `move.uci()`.
- Assuming the agent is always White or ignoring the side to move in the FEN.
- Failing to handle quiet positions, castling, en passant, or promotion.
- Using randomness that does not derive from the supplied seed.
- Spending nearly the entire clock on one early move.
- Requiring constructor arguments or defining multiple eligible Agent classes.
- Importing another Agent class into the submission file.
- Depending on an extra file, package, model, engine, network service, thread, or subprocess.
- Testing only as one color.
- Treating a public practice score as the official class rank.

11. Submission checklist

- ☐ My file is named with my actual NetID, for example `abc123.py`.
- ☐ It is no larger than 1 MiB.
- ☐ It defines exactly one no-argument `ChessAgent` subclass.
- ☐ `choose_move` always returns a legal UCI string.
- ☐ I tested the agent as both White and Black.
- ☐ I use only approved dependencies.
- ☐ I use no engine, network, subprocess, or extra file.
- ☐ All randomness derives from the provided seed.
- ☐ The web page shows `READY TO SUBMIT`, or the CLI prints `PASSED — SUBMISSION READY`.
- ☐ I inspected the local fault and timing report.
- ☐ I saved the exact file I uploaded to Gradescope.
- ☐ I acknowledged all authorized borrowed ideas and code.

Keep the exact submitted source file for your report and presentation. Official results correspond to the frozen Gradescope submission, not to a later local revision.